DESIGN DOCUMENT

Development of an Ethernet-Based Vehicle Monitoring System Using Linux Socket Programming

Project Technical Documentation

1. Introduction

Modern vehicles contain multiple Electronic Control Units (ECUs) that exchange information to monitor vehicle operation. Communication between ECUs is important for sharing parameters such as vehicle speed, engine RPM and steering angle.

This project demonstrates communication between two simulated ECUs using Linux socket programming in the C programming language. The Vehicle ECU generates simulated vehicle parameters and sends them to the Dashboard ECU using TCP or UDP over an IP network.

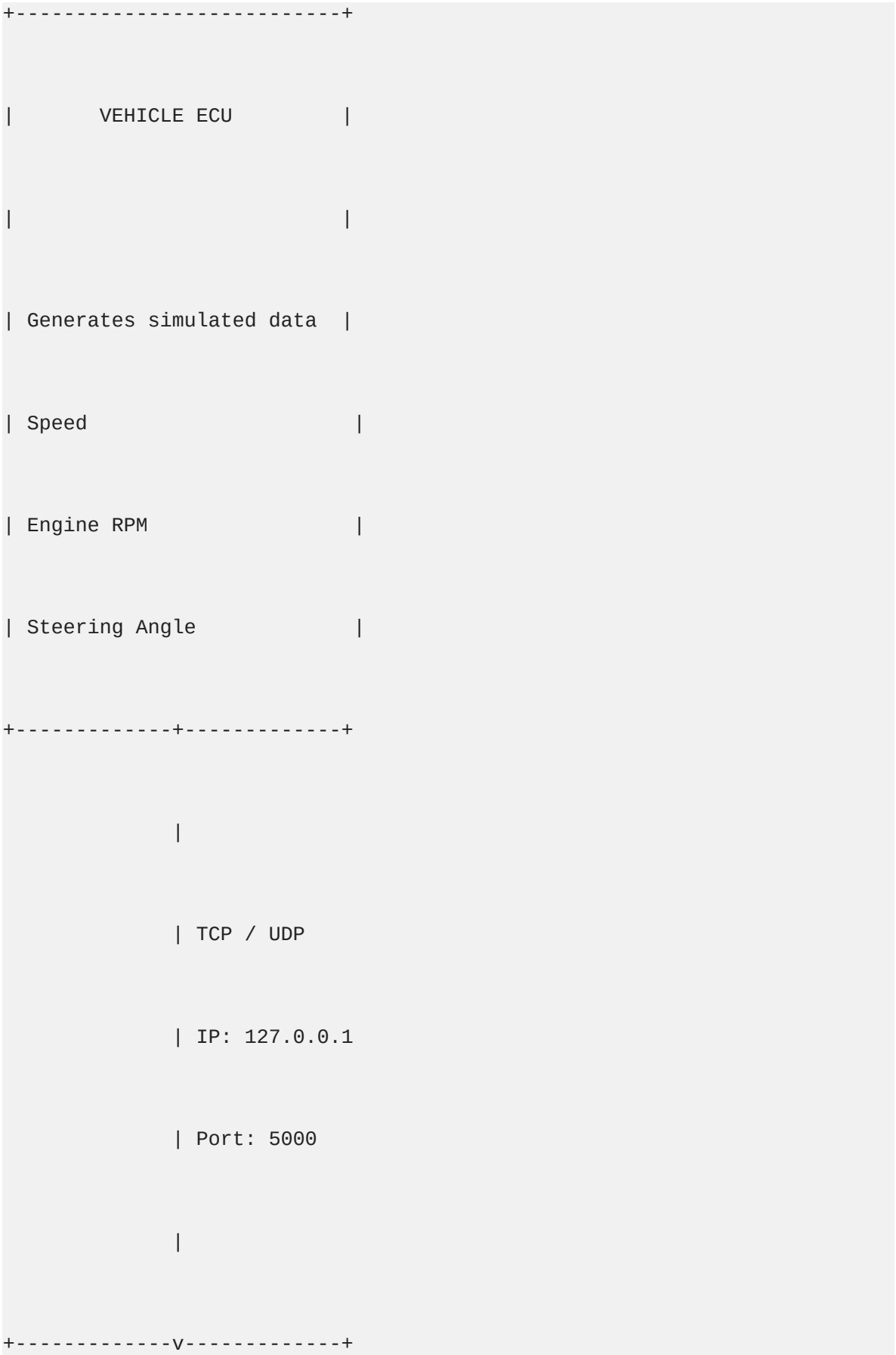
2. Problem Statement

To develop a Linux-based vehicle monitoring system using socket programming in which a simulated Vehicle ECU transmits vehicle parameters to a Dashboard ECU. The system must support TCP and UDP communication and allow their communication behaviour to be compared.

3. Objectives

- Understand Linux socket programming.
- Implement client-server communication using TCP.
- Implement connectionless communication using UDP.
- Generate simulated vehicle parameters periodically.
- Receive and display vehicle data on a dashboard terminal.
- Capture and analyse network packets using Wireshark/TShark.
- Compare TCP and UDP communication.

4. System Architecture



```
|      DASHBOARD ECU         |

|                            |

| Receives vehicle data      |

| Displays parameters        |

+----------------------------+
```

The Vehicle ECU acts as the sender/client. The Dashboard ECU acts as the receiver. For TCP, the Dashboard ECU listens for and accepts a connection. For UDP, it binds a socket and waits for datagrams.

5. Software Requirements

Component	Details
Operating System	Ubuntu 22.04.5 LTS under WSL2
Programming Language	C
Compiler	GCC 11.4.0
Build Tool	GNU Make 4.3
Network Analysis	Wireshark / TShark 3.6.2
Communication	Linux IPv4 sockets using TCP and UDP

6. Vehicle Data Structure

Both programs use the following C structure to represent the simulated vehicle information:

```
struct VehicleData

{

    int speed;

    int engine_rpm;

    float steering_angle;

};
```

Parameter	Unit	Generation Range
Vehicle Speed	km/h	0 to 120
Engine RPM	RPM	800 to 5000
Steering Angle	degrees	-45.0 to +45.0

The structure occupies 12 bytes on the tested system. Both programs use the same structure definition and transmit its contents directly.

7. Vehicle ECU Implementation

The Vehicle ECU is implemented in **src/vehicle_ecu.c**. It accepts a command-line

argument to select either TCP or UDP mode.

1. Validates the selected communication mode.
2. Creates an IPv4 socket using SOCK_STREAM for TCP or SOCK_DGRAM for UDP.
3. Configures the Dashboard ECU address as 127.0.0.1 and port 5000.
4. Establishes a connection when TCP mode is selected.
5. Generates random vehicle parameters.
6. Transmits the structure using send() for TCP or sendto() for UDP.
7. Repeats transmission at one-second intervals.

8. Dashboard ECU Implementation

The Dashboard ECU is implemented in **src/dashboard_ecu.c**. It also accepts a command-line argument to select TCP or UDP mode.

1. Creates the appropriate IPv4 socket.
2. Binds the socket to port 5000.
3. In TCP mode, listens for an incoming connection and accepts the Vehicle ECU client.
4. In UDP mode, waits for incoming datagrams using recvfrom().
5. Receives the vehicle data structure.
6. Displays speed, engine RPM and steering angle in the terminal.
7. Handles connection closure or reception errors.

9. TCP Communication Flow

```
Dashboard ECU                 Vehicle ECU

    |                             |

    |---- socket() ---------------|

    |---- bind()                  |
```

```
    |---- listen()                |

    |                             |---- socket()

    |                             |---- connect()

    |<--------- accept() ---------|

    |                             |

    |<--------- send() -----------|

    |---------- recv() ---------->|

    |                             |

    |       Data exchange         |

    |                             |

    |<--------- FIN/close --------|
```

TCP is connection-oriented. It establishes a connection before application data is exchanged and provides reliable, ordered byte-stream delivery.

10. UDP Communication Flow

```
Dashboard ECU                 Vehicle ECU

    |                             |
```

```
    |---- socket()                  |

    |---- bind()                    |

    |                              |---- socket()

    |                              |

    |<--------- sendto() ----------|

    |---------- recvfrom() ------->|

    |                              |

    |       Datagram exchange      |
```

UDP is connectionless. It sends individual datagrams without establishing a connection. It does not guarantee delivery or provide application-level confirmation that the receiver received the data.

11. Network Configuration

Parameter	Configuration
Address Family	AF_INET (IPv4)
Dashboard IP	127.0.0.1
Port	5000
TCP Socket Type	SOCK_STREAM
UDP Socket	SOCK_DGRAM

Type	
Test Interface	Linux loopback interface (lo)

The current implementation is tested on the local loopback interface. It demonstrates IP socket communication but has not been tested between separate physical ECUs or over an external Ethernet cable.

12. Build and Execution

Both programs are compiled from the project root using:

```
make -f src/Makefile
```

UDP mode:

```
Terminal 1: ./dashboard_ecu udp


Terminal 2: ./vehicle_ecu udp
```

TCP mode:

```
Terminal 1: ./dashboard_ecu tcp


Terminal 2: ./vehicle_ecu tcp
```

13. Packet Analysis

Wireshark/TShark was used to capture and inspect traffic on the loopback interface. The UDP capture contains datagrams addressed to port 5000. The TCP capture shows connection establishment, data segments, acknowledgements and connection termination.

The saved captures are **wireshark/udp_capture.pcapng** and

wireshark/tcp_capture.pcapng.

14. Conclusion

The project demonstrates simulated vehicle monitoring through Linux socket programming. Both TCP and UDP modes successfully transfer vehicle speed, engine RPM and steering angle from the Vehicle ECU to the Dashboard ECU. Packet captures provide evidence of the communication behaviour of both protocols.